

Milestone 2 Progress Summary

Technical Architecture and UX Design, Economic Development in Latin America (App Phase)

1. Define the MVP and Feature Priorities

Must have features:

- Overview page with headline numbers and trend charts for GDP per capita, unemployment, and inflation.
- Social Development page with life expectancy, poverty rate, and school enrollment.
- Investment, Technology and Demographics page with foreign investment, internet access, and population.
- Country filter, year range filter, and indicator selector.
- Correct handling of missing data, shown as No data available rather than zero or blank.
- Argentina's inflation shown separately from the other countries due to its scale.

Nice to have features:

- Insights panel with the 16 documented insights, browsable by theme.
- Data table view showing the raw values behind any chart.
- CSV export option.
- About and methodology page linking back to the White Paper and documentation.

Postponed to a later phase:

- The 15 additional data sources reviewed earlier.
- Live or real-time data updates.
- Forecasting or predictive features.
- User accounts, saved views, or personalization.

Main user goal:

To explore and compare the 12 confirmed economic and social indicators across the 6 Latin American countries, and understand development trends between 2014 and 2024, without any predictive or causal claims.

2. Define Application Architecture

The application uses a simplified architecture suited to a static, descriptive dataset with no live data or user accounts.

- Frontend and backend: combined in a single Python application built with Streamlit.
- Data layer: the existing cleaned CSV file, loaded directly using pandas at runtime.
- Database: not needed for the first version, since the dataset is small (792 rows) and static.
- API: not needed for the first version, since there is no external live data source yet.
- Hosting and deployment: Streamlit Community Cloud, deploying directly from the existing GitHub repository.

Simple data flow: cleaned CSV, loaded by pandas, filtered by the app, rendered by Streamlit, shown to the user's browser. A separate architecture diagram was attached to this task in Notion.

3. Select Technology Stack

- Frontend framework: Streamlit, chosen because it turns Python data-analysis code directly into an interactive web app.
- Backend framework: not separate, handled within the same Streamlit app.
- Database: none for the first version, data loads directly from the cleaned CSV using pandas.
- Data visualization library: Plotly, used through Streamlit, chosen for interactive charts with simple Python code.
- Hosting and deployment: Streamlit Community Cloud, deploying for free directly from the existing GitHub repository.
- Other tools: pandas, already used during the cleaning and analysis phase.

4. Define Data Flow

- Data is stored as the cleaned CSV file inside the project's GitHub repository, alongside the app's code.
- Data is processed by pandas, loaded once into memory when the app starts; no extra cleaning is needed.
- The frontend retrieves data directly from memory, since Streamlit combines the frontend and data logic.
- Filters narrow the in-memory data to the user's selection; nothing is reloaded from the file itself.
- Missing data is shown as No data available rather than zero or a blank space.

5. Create Application Wireframes

Six low-fidelity wireframes were created, one for each required page: Home/Overview, Social Development, Investment and Technology, Insights, Data Table, and About/Methodology. Each wireframe shows the same navigation bar for moving between pages, and most pages include the country, year, and indicator filters near the top. The layout follows the structure already used in the Power BI dashboard. The six wireframe files were attached to this task in Notion.

6. Define User Interactions

- Country filter: selecting countries updates all charts and the table on the current page immediately.
- Year range filter: selecting a start and end year updates the charts and table to that range.
- Indicator selector: choosing an indicator redraws the chart with that indicator's data.
- Chart interactions: hovering over a point shows the exact value, country, and year in a tooltip.
- Data table interactions: clicking a column header sorts the table by that column.
- CSV export: clicking Export CSV downloads exactly the rows currently shown after filtering.
- Navigation: clicking a page name switches pages while keeping the same filters selected.

7. Review Technical and UX Decisions

Technical risks:

- Streamlit Community Cloud is a free hosting tier and may have limits on uptime or traffic under heavy use.
- There is no database in this version, so significant data growth or live updates would need rework later.
- The cleaned CSV is loaded fully into memory each run, fine at the current size but not at a much larger scale.
- The wireframes are layout only; colors, branding, and mobile view have not been decided yet.

Unresolved questions:

- Should the app pull live data from the World Bank API later, or keep using the static dataset for now?
- Should any of the 15 additional data sources be added later, and if so, which ones come first?
- Is there a target audience or launch date already decided that should shape the app's design?

Dependencies:

- Deployment depends on Streamlit Community Cloud remaining available and free to use.
- The app depends on the GitHub repository staying the single source of truth for code and data.

Information or support needed from the Foundation:

- Confirmation of the target audience and any branding or style guidelines for the visual design phase.
- A decision on whether live data is expected in this first version or can wait for a later phase.
- Confirmation that Streamlit Community Cloud is an acceptable hosting choice.

Proposed changes to MVP scope: none at this time. The scope defined earlier still holds, pending the Foundation's answers to the open questions above.

This summary covers all seven tasks of Milestone 2. Supporting files (architecture diagram and wireframes) were attached to their respective tasks in Notion.